

1.Explain how to satisfy a request of size n from a list of free holes. Explain First Fit, Best Fit and Worst Fit.

Dynamic Storage Allocation Problem:

When memory is allocated using the variable partition scheme, free memory is available as holes of different sizes. Whenever a process requests memory of size n , the operating system must find a suitable free hole to allocate memory.

This problem is called the Dynamic Storage Allocation Problem.

The Dynamic Storage Allocation Problem concerns how to satisfy a request of size n from a list of free holes available in memory.

The operating system uses three allocation strategies:

1. First Fit
2. Best Fit
3. Worst Fit

1. First Fit:

First Fit allocates the first hole that is large enough to satisfy the memory request.

Working-

- Search begins from the beginning of the free-hole list (or from the previous search position).
- As soon as a suitable hole is found, memory is allocated.
- The remaining part of the hole is returned to the free-hole list.

Advantages-

- Fastest allocation technique.
- Requires less searching.

Disadvantages-

- Creates external fragmentation.
- Small holes accumulate near the beginning of memory.

2. Best Fit:

Best Fit allocates the smallest hole that is large enough for the requested memory.

Working-

- Entire free-hole list must be searched.
- Select the smallest suitable hole.
- Leaves the smallest leftover space.

Advantages-

- Minimizes unused space after allocation.
- Better utilization of memory.

Disadvantages-

- Requires searching the entire list.
- More time-consuming than First Fit.
- Produces many very small unusable holes, leading to external fragmentation.

3. Worst Fit:

Worst Fit allocates the largest available hole for the requested process.

Working-

- Entire list is searched.
- Largest hole is selected.
- Remaining space is still large enough for future allocations.

Advantages-

- Produces large leftover holes.
- Large remaining holes may be useful for future processes.

Disadvantages-

- Requires complete search.
- Wastes large memory blocks.
- Less efficient than First Fit in practice.

Comparison Table:-

First Fit	Best Fit	Worst Fit
Allocates first suitable hole	Allocates smallest suitable hole	Allocates largest hole
Faster	Slower	Slower
Stops after first suitable hole	Searches entire list	Searches entire list
Moderate memory utilization	Better memory utilization	Poor memory utilization
Causes external fragmentation	Causes external fragmentation	Causes external fragmentation

2. Describe Memory Allocation: Contiguous Memory Allocation, Fragmentation, and Compaction.

Memory Allocation:

Memory allocation is the process of allocating main memory to processes for execution.

In Contiguous Memory Allocation, each process occupies one continuous block of physical memory.

There are two types of contiguous memory allocation:

1. Fixed Partition Scheme (MFT)
2. Variable Partition Scheme (MVT)

1. Fixed Partition Scheme (MFT)

In this scheme, the main memory is divided into several fixed-size partitions.

Features-

- Each partition contains only one process.
- Degree of multiprogramming depends on the number of partitions.
- When a partition becomes free, another process from the input queue is loaded.
- Originally used in IBM OS/360 operating system.

Advantages-

- Simple to implement.
- Easy memory management.

Disadvantages-

- Causes Internal Fragmentation.
- Degree of multiprogramming is limited.

2. Variable Partition Scheme (MVT)

In this scheme, partitions are created dynamically according to the size of the process.

Working-

- Initially, all memory is one large free block called a hole.
- When a process arrives, the operating system searches for a suitable hole.
- If the hole is larger than required, it is split into two parts.
- When a process terminates, its memory is released.
- Adjacent free holes are merged to form a larger hole.

Advantages-

- Better utilization of memory.
- Supports processes of different sizes.

Disadvantages-

- Causes External Fragmentation.
- Needs dynamic storage allocation algorithms (First Fit, Best Fit, Worst Fit).

Fragmentation

Fragmentation is the wastage of memory that occurs during memory allocation.

There are two types:

1. Internal Fragmentation
2. External Fragmentation

Internal Fragmentation:

Internal Fragmentation is the unused memory inside an allocated partition.

Cause

- Memory is allocated in fixed-size blocks.
- If the allocated block is larger than the requested memory, the remaining space is wasted.

Example

Suppose a partition is 18,464 bytes and the process requires 18,462 bytes.

Unused memory = 2 bytes

This unused space inside the partition is called Internal Fragmentation.

External Fragmentation:

External Fragmentation occurs when free memory is divided into many small non-contiguous holes, even though the total free memory is sufficient.

Cause

- Processes are repeatedly loaded and removed.
- Free memory becomes scattered into many small holes.

Disadvantages

- Large processes cannot be allocated memory.
- Memory utilization decreases.

Compaction:

Compaction is the process of combining scattered free memory blocks into one large continuous block.

Working

- The operating system moves all processes towards one end of memory.
- All free holes are merged into one large hole.

Requirements

Compaction is possible only when:

- Relocation is dynamic.
- Address binding is performed at execution time.

If relocation is static (compile time or load time), compaction cannot be performed.

Advantages

- Eliminates external fragmentation.
- Produces one large free memory block.
- Improves memory utilization.

Disadvantages

- Expensive because all processes have to be moved.
- Takes considerable execution time.

Difference between Internal and External Fragmentation

Internal Fragmentation	External Fragmentation
Wastage occurs inside allocated partition	Wastage occurs between allocated partitions
Common in fixed partition allocation	Common in variable partition allocation
Cannot be removed by compaction	Can be removed by compaction

3.Explain the Hardware of Paging with a neat diagram and example.

Hardware of Paging:

Paging is a memory management technique that allows the physical address space of a process to be non-contiguous. It eliminates external fragmentation and removes the need for compaction.

Paging is implemented through the cooperation of the Operating System and Computer Hardware.

In paging,

- Physical memory is divided into fixed-size blocks called Frames.
- Logical memory is divided into fixed-size blocks called Pages.
- Frame size = Page size.
- A process is divided into pages, and these pages can be loaded into any available frames in physical memory.

Hardware Components

The hardware required for paging consists of:

1. Page Table

- Each process has its own Page Table.
- It stores the frame number corresponding to each page.
- Only the page table of the currently executing process is active.

2. Page Table Base Register (PTBR)

- The PTBR points to the starting address of the page table in main memory.
- It helps the CPU locate the correct page table.

3. Frame Table

- Maintains information about all frames in physical memory.
- Indicates whether a frame is free or allocated.

4. Memory Management Unit (MMU)

The MMU translates the logical address generated by the CPU into a physical address.

Address Translation

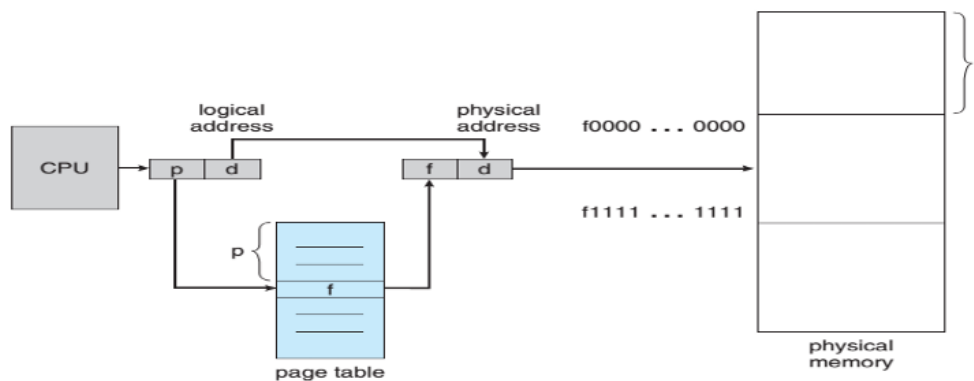
Every logical address generated by the CPU is divided into two parts:

- Page Number (p)
- Page Offset (d)

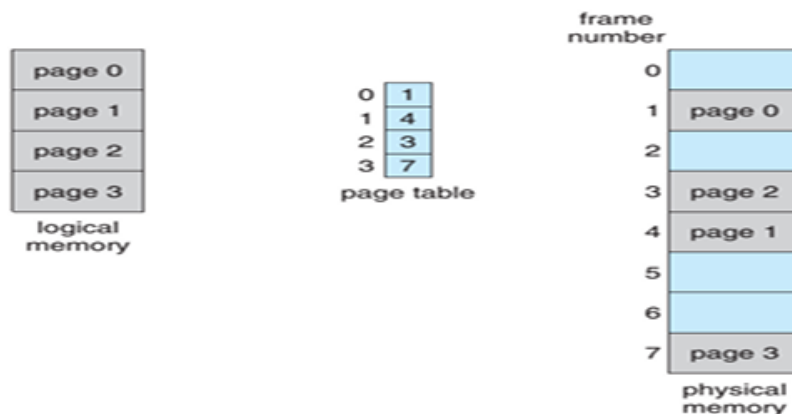
Translation Steps

1. CPU generates a logical address (p, d).
2. The page number (p) is used as an index into the page table.
3. The page table provides the corresponding frame number (f).
4. The frame number is combined with the page offset (d).
5. The resulting address (f, d) becomes the physical address sent to memory.

Diagram:



The below shows the paging of Logical and Physical memory:



Example

Assume:

- Page size = 4 bytes
- Logical Address = 13

Step 1

Logical Address = 13

Page Number = 3

Offset = 1

Step 2

From the Page Table,

Page 3 → Frame 2

Step 3

Physical Address

= Frame Number + Offset

= (2,1)

Thus, the logical address is translated into the corresponding physical address.

Advantages of Paging

- Eliminates External Fragmentation.
- No need for Compaction.
- Efficient memory utilization.
- Allows non-contiguous memory allocation.
- Supports virtual memory.

Disadvantage

Paging causes Internal Fragmentation because memory is allocated in fixed-size pages and frames.

4.Explain Hardware Support for Paging with TLB (Translation Lookaside Buffer).

Hardware Support for Paging with TLB

In a paging system, every logical address generated by the CPU must first access the Page Table to obtain the frame number and then access the required memory location.

Thus, two memory accesses are needed for every memory reference:

1. Access the Page Table.
2. Access the actual memory location.

This doubles the memory access time and slows down the system. To overcome this problem, a Translation Lookaside Buffer (TLB) is used.

Translation Lookaside Buffer (TLB):

A Translation Lookaside Buffer (TLB) is a special, small, fast associative memory (cache) that stores recently used page table entries.

Its purpose is to reduce address translation time and improve memory access speed.

Hardware Support using TLB

The hardware consists of:

- CPU
- TLB (Associative Memory)
- Page Table
- Main Memory
- Memory Management Unit (MMU)

The TLB stores only a few frequently used page-table entries.

Working of TLB

Case 1: TLB Hit

1. CPU generates a logical address (Page Number, Offset).
2. The page number is searched in the TLB.
3. If found (TLB Hit), the corresponding frame number is obtained immediately.
4. The frame number is combined with the offset.
5. The physical address is generated and memory is accessed.

Only one memory access is required.

Case 2: TLB Miss

1. CPU generates a logical address.
2. The page number is searched in the TLB.
3. If not found (TLB Miss), the page table in main memory is accessed.
4. The frame number is obtained from the page table.
5. The page number and frame number are inserted into the TLB.
6. The physical address is generated.
7. The required memory location is accessed.

Two memory accesses are required.

TLB Structure

Each TLB entry contains:

- Key (Tag) – Page Number
- Value – Frame Number

The size of a TLB generally ranges from 32 to 1024 entries.

If the TLB becomes full, an existing entry is replaced using a page replacement algorithm.

Address Space Identifier (ASID)

Each TLB entry stores an Address Space Identifier (ASID).

- ASID uniquely identifies a process.
- It provides address-space protection.
- It allows the TLB to store entries of multiple processes simultaneously.
- If ASIDs are not supported, the TLB must be flushed whenever a new process is loaded.

TLB Hit Ratio and Miss Ratio

- Hit Ratio: Percentage of times the required page number is found in the TLB.
- Miss Ratio: Percentage of times the required page number is not found.

Formula:

$$\text{TLB Miss Ratio} = 1 - \text{TLB Hit Ratio}$$

Effective Memory Access Time (EAT)

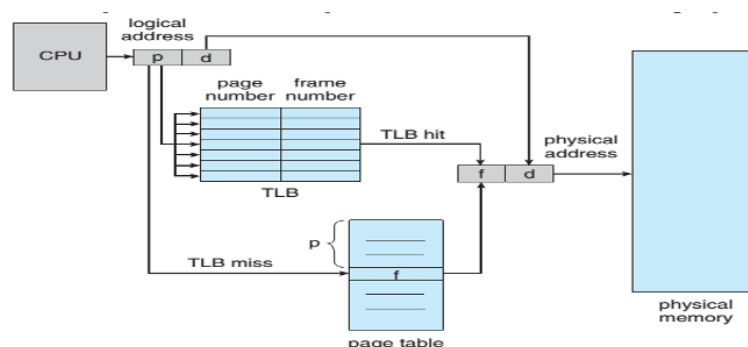
Effective Memory Access Time is the average time required to access memory, considering both TLB hits and TLB misses.

It depends on:

- TLB Hit Ratio
- TLB Miss Ratio
- Memory Access Time

A higher TLB Hit Ratio results in a lower Effective Memory Access Time.

Diagram:



Advantages of TLB:

- Reduces memory access time.
- Improves system performance.
- Reduces page table accesses.
- Increases CPU efficiency.
- Provides faster address translation.

5. Explain the Structure of Page Table.

Structure of Page Table:

A Page Table is a data structure maintained by the operating system that maps the logical pages of a process to the physical frames in main memory.

Each process has its own page table. The Page Table Base Register (PTBR) points to the starting address of the currently active page table.

Need for Page Table:

In paging,

- Logical memory is divided into Pages.
- Physical memory is divided into Frames.
- CPU generates a logical address.
- The Memory Management Unit (MMU) uses the page table to translate the logical address into a physical address.

Thus, the page table enables logical-to-physical address translation.

Structure and Working of Page Table

Each page table contains one entry for every page of a process.

A page table entry generally contains:

- Frame Number
- Valid/Invalid Bit
- Protection Bit
- Reference Bit
- Dirty (Modify) Bit

Working

1. CPU generates a logical address (Page Number, Offset).
2. The Page Number acts as an index into the Page Table.
3. The corresponding Frame Number is obtained.
4. The Frame Number is combined with the Offset.
5. The physical address is generated and sent to main memory.

Types (Structure) of Page Tables:

As the logical address space becomes very large, a single page table occupies a huge amount of memory. To overcome this problem, different page table organizations are used.

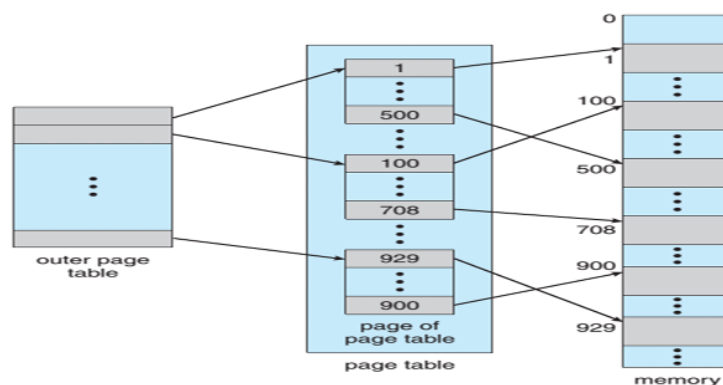
They are:

1. Hierarchical Page Table
2. Hashed Page Table
3. Inverted Page Table

1. Hierarchical Page Table (Two-Level Paging):

Hierarchical Paging divides a large page table into smaller page tables.

The page table itself is divided into pages. Therefore, address translation takes place in two levels.



Working:

For a 32-bit logical address with 4 KB page size:

- Logical address contains:
 - Outer Page Number (P_1)
 - Inner Page Number (P_2)
 - Offset
- P_1 indexes the outer page table.
- P_2 indexes the inner page table.
- The frame number is obtained and combined with the offset to generate the physical address.

This method is also known as Forward Mapped Page Table.

Advantages

- Reduces memory required for storing page tables.
- Suitable for large logical address spaces.

Disadvantages

- Requires extra memory access.
- Address translation becomes slower.

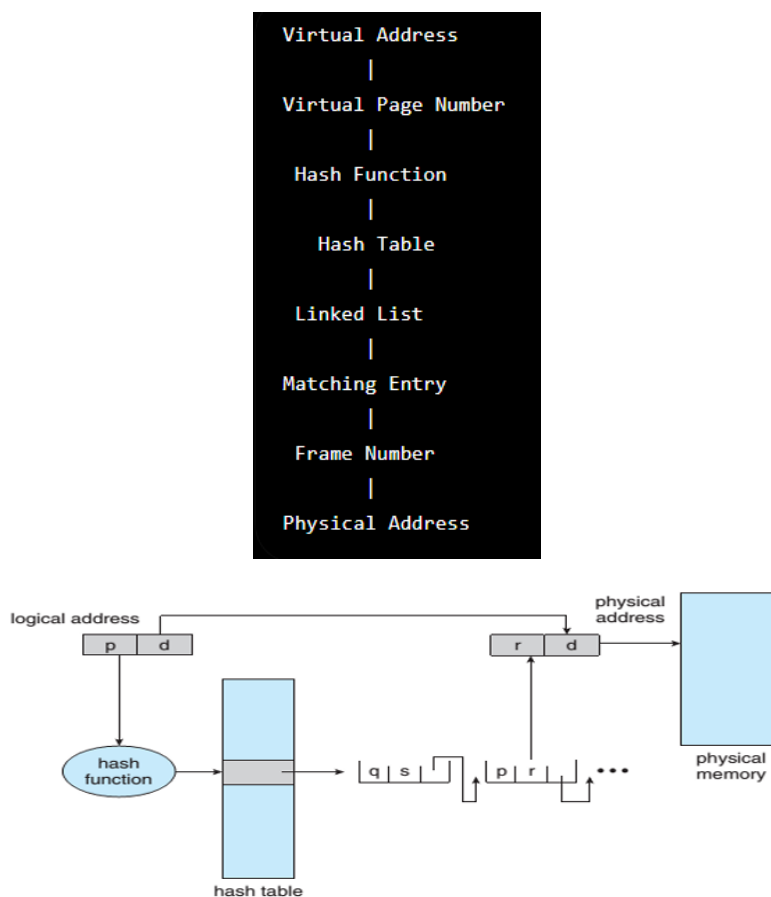
2. Hashed Page Table:

Hashed Page Tables are used in systems having address spaces greater than 32 bits.

A hash function is applied to the Virtual Page Number (VPN) to locate the required page table entry.

Working

1. CPU generates a virtual address.
2. The Virtual Page Number is passed through a hash function.
3. The hash function points to an entry in the hash table.
4. Each hash table entry points to a linked list.
5. The operating system searches the linked list for a matching Virtual Page Number.
6. If a match is found, the corresponding Frame Number is obtained.
7. The Frame Number and Offset together form the Physical Address.



Advantages

- Faster lookup for large address spaces.
- Efficient for 64-bit systems.
- Reduces page table search time.

Disadvantages

- Hash collisions may occur.
- Collision resolution increases lookup time.
- More complex implementation.

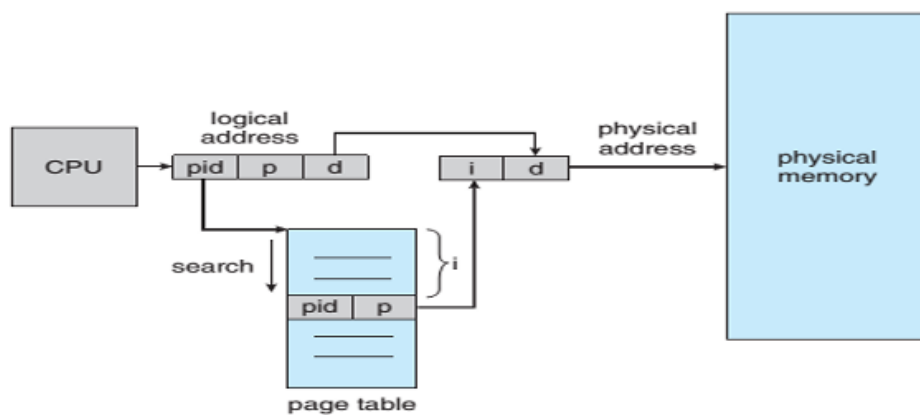
3. Inverted Page Table:

An Inverted Page Table contains one entry for each physical frame instead of one entry for each logical page.

Thus, only one page table exists in the entire system.

Each entry stores:

- Process ID (Address Space Identifier)
- Page Number



Working:

The virtual address consists of:

<Process ID, Page Number, Offset>

1. The pair (Process ID, Page Number) is searched in the inverted page table.
2. If a matching entry is found, its index gives the Frame Number.
3. The Frame Number is combined with the Offset to form the Physical Address.
4. If no match is found, an illegal address access occurs.

Advantages

- Requires much less memory.
- Only one page table is maintained.
- Suitable for large address spaces.

Disadvantages

- Searching takes more time.
- Difficult to implement shared memory.

Advantages of Page Table

- Supports logical-to-physical address translation.
- Enables non-contiguous memory allocation.
- Eliminates external fragmentation.
- Provides memory protection.
- Supports virtual memory.

6.How can Protection and Sharing be achieved in Paging?

Protection and Sharing in Paging:

Paging is a memory management technique in which:

- Logical memory is divided into Pages.
- Physical memory is divided into Frames.
- Each process has its own Page Table.

Besides memory allocation, paging also provides memory protection and memory sharing through hardware support and page table entries.

Memory Protection in Paging:

Memory Protection is a mechanism that prevents a process from accessing or modifying the memory belonging to another process or the operating system.

Protection is achieved by using Page Table Entry (PTE) bits.

Protection using Valid–Invalid Bit

Each Page Table Entry contains a Valid–Invalid Bit.

Valid Bit (V)-

- Indicates that the page belongs to the process.
- Access to the page is permitted.

Invalid Bit (I)-

- Indicates that the page does not belong to the process.
- Any attempt to access the page causes a trap to the operating system.

Thus, the Valid–Invalid Bit protects a process from accessing illegal memory locations.

Protection using Protection Bits:

Each page table entry also contains Protection Bits.

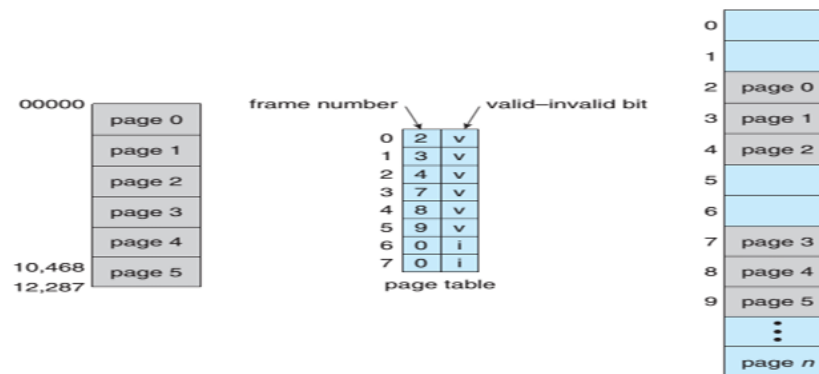
These specify the type of access allowed for a page.

Examples:

- Read Only (R)
- Read and Write (R/W)
- Execute (X)

If a process performs an operation that is not permitted by the protection bits, the hardware generates a protection fault and transfers control to the operating system.

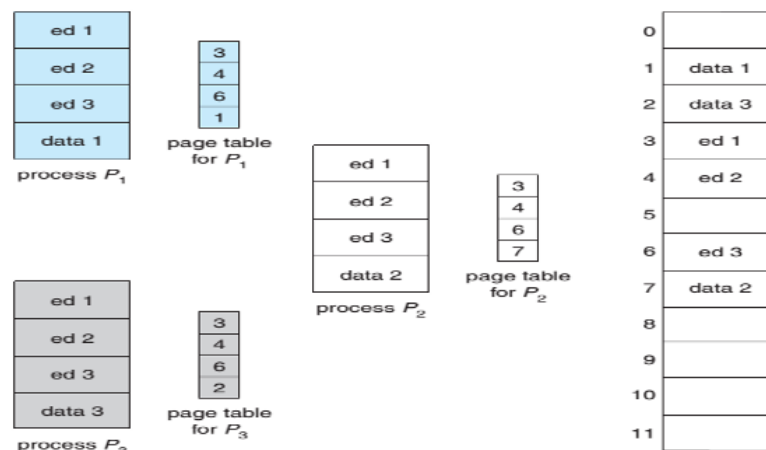
Diagram:



Memory Sharing in Paging:

Paging allows multiple processes to share common pages in physical memory.

Instead of keeping multiple copies of the same program, several processes can refer to the same physical frame.



Working

- Different page tables may point to the same frame.
- Shared pages are generally read-only.
- Since the pages cannot be modified, all processes safely use the same copy.

Examples include:

- Shared libraries
- Compiler routines
- System libraries

Advantages of Sharing

- Reduces memory usage.
- Saves physical memory.
- Improves system efficiency.
- Multiple processes can execute the same program simultaneously.
- Reduces loading time.

Protection and Sharing Together

Paging achieves both protection and sharing through the Page Table.

- Valid-Invalid Bit ensures that only legal pages are accessed.
- Protection Bits control read, write and execute permissions.
- Multiple page tables can point to the same physical frame to enable safe sharing.

Advantages of Protection and Sharing

- Prevents unauthorized memory access.
- Protects operating system memory.
- Protects one process from another.
- Saves memory by sharing common code.
- Improves overall system performance.

7.What is a Page Fault? Explain the steps involved in handling a Page Fault.

Page Fault:

A Page Fault occurs when a process tries to access a page that is not currently present in the main memory.

When this happens, the operating system interrupts the process, loads the required page from secondary storage into main memory, updates the page table, and then resumes the execution of the process.

Steps in Handling a Page Fault

When a page fault occurs, the operating system performs the following steps:

Step 1: Check the Page Reference

- The CPU generates a logical address.
- The page table is checked.
- If the page is not present, a page fault trap occurs.

Step 2: Verify the Memory Reference

- The operating system checks whether the memory reference is valid or invalid.
- If the reference is invalid, the process is terminated.

- If it is valid, the operating system locates the required page on the backing store (disk).

Step 3: Find a Free Frame

The operating system searches for a free frame in main memory.

- If a free frame is available, it is allocated.
- If no free frame is available, a Page Replacement Algorithm is used to select a victim page.

Step 4: Read the Required Page

- The required page is read from the backing store (disk).
- It is loaded into the selected free frame.

Step 5: Update the Page Table

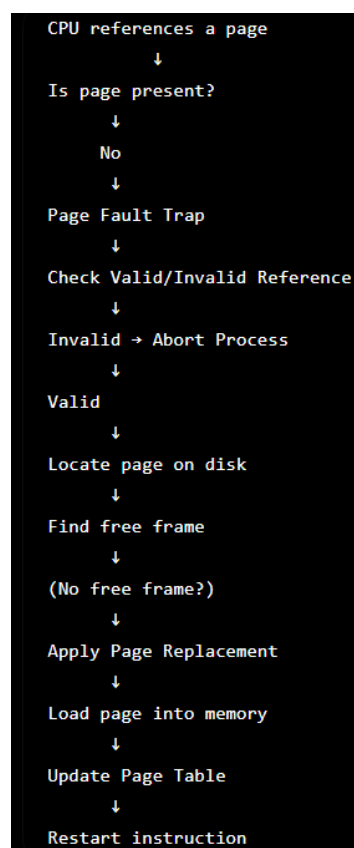
After loading the page,

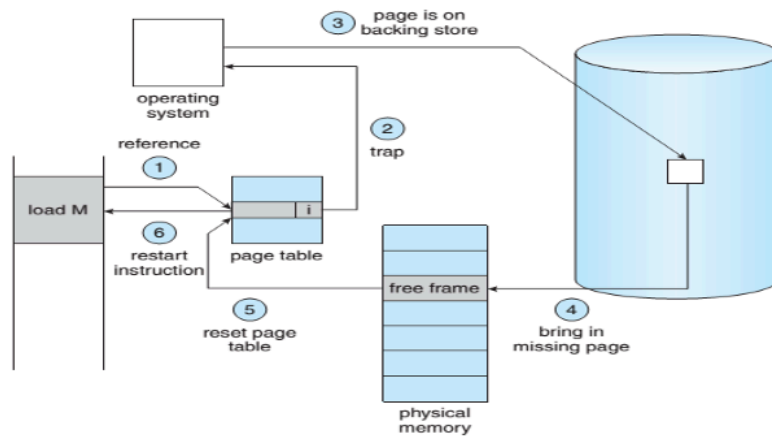
- The page table entry is updated.
- The frame number is entered.
- The valid/invalid bit is changed to Valid.

Step 6: Restart the Process

- The interrupted instruction is restarted.
- The process continues execution normally.

Flow of Page Fault Handling





Advantages

- Enables execution of programs larger than physical memory.
- Supports virtual memory.
- Improves memory utilization.
- Allows demand paging.

Disadvantages

- Frequent page faults reduce system performance.
- Excessive page faults may lead to thrashing.
- Disk access during page faults increases execution time.

8.What is Thrashing? Explain the Working Set Model and Page Fault Frequency (PFF).

Thrashing:

Thrashing is a condition in which a process spends more time paging than executing.

It occurs when the number of frames allocated to a process is insufficient to hold the pages that are actively in use. As a result, page faults occur repeatedly, and the CPU utilization decreases.

Cause of Thrashing-

The operating system monitors CPU utilization.

- When CPU utilization decreases, the operating system increases the degree of multiprogramming by introducing new processes.
- If a global page replacement algorithm is used, the new process starts taking frames from other processes.
- Those processes then experience page faults and take frames from other processes.
- This results in continuous paging activity.
- The paging device becomes busy, while the CPU remains idle.
- Consequently, CPU utilization decreases further.

- The operating system again increases the degree of multiprogramming, worsening the situation.

Thus, the system enters a state called Thrashing.

Effects of Thrashing:

- CPU utilization decreases.
- Page fault rate increases rapidly.
- Effective memory access time increases.
- System throughput decreases.
- Processes spend most of their time swapping pages instead of executing.

Solutions to Thrashing:-

According to the notes, thrashing can be controlled using:

1. Local Replacement Algorithm (Priority Replacement Algorithm)
2. Working Set Model (Locality Model)
3. Page Fault Frequency (PFF)

Working Set Model (Locality Model):

The Working Set Model is based on the principle of locality of reference.

A working set is the set of pages that a process is actively using during a particular period of execution.

Working-

- As a process executes, it moves from one locality to another.
- Each locality contains a group of pages that are frequently accessed together.
- The operating system allocates enough frames to hold the entire working set.
- After all pages of the working set are loaded into memory, page faults are greatly reduced.
- When the process moves to another locality, the working set changes, and the operating system allocates frames accordingly.

Thus, by ensuring that every process has enough frames for its current working set, thrashing can be prevented.

Advantages of Working Set Model

- Prevents thrashing.
- Reduces page faults.
- Improves CPU utilization.
- Improves system throughput.

Page Fault Frequency (PFF):

Page Fault Frequency (PFF) is a method that controls thrashing by monitoring the page fault rate of each process.

Working

The operating system continuously observes the page fault frequency.

Case 1: High Page Fault Frequency

- Indicates that the process has too few frames.
- Additional frames are allocated to the process.

Case 2: Low Page Fault Frequency

- Indicates that the process has more frames than required.
- Some frames are removed and allocated to other processes.

Thus, the operating system maintains the page fault rate within acceptable limits, preventing thrashing.

Advantages of PFF

- Controls page fault rate.
- Allocates frames dynamically.
- Prevents thrashing.
- Improves memory utilization.

Difference between Working Set Model and PFF

Working Set Model	Page Fault Frequency (PFF)
Based on locality of reference	Based on page fault rate
Maintains the current working set of pages	Monitors page fault frequency
Allocates frames according to locality	Allocates or removes frames based on fault rate
Prevents thrashing using locality	Prevents thrashing using page fault monitoring

9.Explain the need for Page Replacement and the steps involved in Page Replacement.

Page Replacement:

Page Replacement is a memory management technique used when a page fault occurs and no free frame is available in main memory.

In this situation, the operating system selects a page currently in memory (called the victim page) and replaces it with the required page from secondary storage.

Need for Page Replacement:

In a demand paging system, a page fault occurs whenever the required page is not present in main memory.

If a free frame is available, the required page is simply loaded into that frame.

However, when all frames are occupied, the operating system cannot load the required page unless one of the existing pages is removed.

Therefore, Page Replacement is required to:

- **Load the required page into memory when no free frame is available.**
- **Continue process execution after a page fault.**
- **Improve memory utilization.**
- **Support virtual memory.**
- **Reduce unnecessary process termination.**

Steps in Page Replacement

When a page fault occurs and there is no free frame, the operating system performs the following steps:

Step 1: Locate the Required Page

- **Find the required page on the backing store (disk).**

Step 2: Find a Free Frame

The operating system checks whether a free frame is available.

- **If a free frame exists, use it.**
- **Otherwise, apply a Page Replacement Algorithm to select a victim frame.**

Step 3: Replace the Victim Page

- **The selected victim page is removed from memory.**
- **If the victim page has been modified (Dirty Bit = 1), it is written back to disk.**
- **Update the Page Table and Frame Table.**

Step 4: Load the Required Page

- **Read the required page from disk.**
- **Load it into the newly freed frame.**

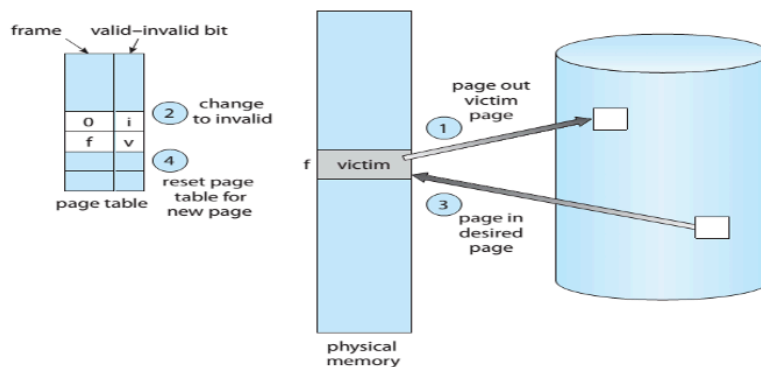
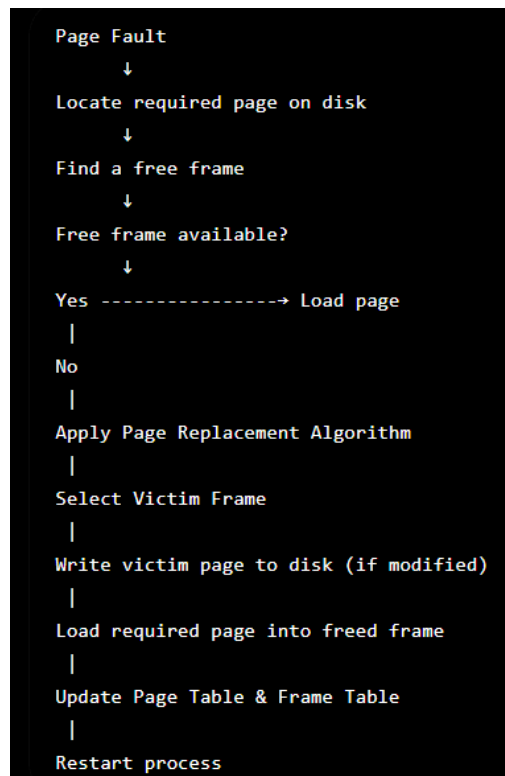
Step 5: Update the Tables

- Update the Page Table.
- Update the Frame Table.
- Mark the page as valid.

Step 6: Continue Execution

- Restart the interrupted instruction.
- The process continues execution normally.

Steps (Flow):



Modify Bit (Dirty Bit)

Each page/frame has a Modify Bit (Dirty Bit) associated with it.

Working

- If the Dirty Bit = 1, the page has been modified after loading into memory.
 - Therefore, it must be written back to disk before replacement.

- If the Dirty Bit = 0, the page has not been modified.
 - Hence, there is no need to write it back to disk, because an identical copy already exists on disk.

Using the Dirty Bit reduces unnecessary disk write operations and improves system performance.

Page Replacement Algorithms

The notes mention the following page replacement algorithms:

1. FIFO (First In First Out)
2. Optimal Page Replacement
3. LRU (Least Recently Used)
4. Counting-Based Algorithms
 - LFU (Least Frequently Used)
 - MFU (Most Frequently Used)

Advantages

- Enables efficient use of limited physical memory.
- Supports virtual memory.
- Allows execution of programs larger than main memory.
- Improves memory utilization.
- Reduces process waiting time.

Disadvantages

- Selecting an inappropriate victim page increases page faults.
- Frequent page replacement may lead to thrashing.
- Disk I/O increases if many modified pages must be written back.

10.Explain the Page Replacement Algorithms: FIFO, Optimal, LRU, MRU, LFU and MFU with examples.

Page Replacement Algorithms

A Page Replacement Algorithm is used when a page fault occurs and no free frame is available in main memory. It selects a victim page to be removed so that the required page can be loaded into memory.

The objective is to minimize the number of page faults and improve system performance.

The commonly used page replacement algorithms are:

1. FIFO
2. Optimal
3. LRU
4. MRU

- 5. LFU
- 6. MFU

1. FIFO (First-In First-Out)

FIFO replaces the page that entered memory first, regardless of how frequently or recently it has been used.

Working

- Maintain pages in a queue.
- When a page fault occurs, remove the oldest page.
- Insert the new page at the rear of the queue.

Advantages

- Simple to implement.
- Low overhead.

Disadvantages

- May replace an important page that is still in use.
- Can suffer from Belady's Anomaly (page faults may increase even when more frames are allocated).

2. Optimal Page Replacement

Optimal Page Replacement replaces the page that will not be used for the longest period in the future.

Working

- Look ahead in the reference string.
- Replace the page whose next use is farthest in the future.

Advantages

- Produces the minimum number of page faults.
- Used as the benchmark for comparing other algorithms.

Disadvantages

- Requires future knowledge of page references.
- Impossible to implement in real operating systems.

3. LRU (Least Recently Used)

LRU replaces the page that has not been used for the longest time in the past.

Working

- Track the recent usage of pages.
- Replace the least recently used page.

The PPT mentions two implementations:

(a) Counter Method

- Every page table entry stores the time of last use.
- Replace the page with the smallest time value.

(b) Stack Method

- Maintain pages in a stack.
- Recently used page stays on the top.
- Least recently used page remains at the bottom and is replaced.

Advantages

- Better performance than FIFO.
- Approximates Optimal replacement.

Disadvantages

- Requires additional hardware or software support.
- More expensive than FIFO.

4. MRU (Most Recently Used)

MRU replaces the page that was used most recently.

Working

- Identify the most recently accessed page.
- Replace that page whenever a page fault occurs.

Advantages

- Suitable for applications where recently used pages are less likely to be used again.

Disadvantages

- Generally performs worse than LRU.
- Not commonly used in practical operating systems.

5. LFU (Least Frequently Used)

LFU replaces the page with the lowest reference count.

Working

- Count the number of times each page is accessed.

- Replace the page having the smallest count.

Advantages

- Frequently used pages remain in memory.

Disadvantages

- Old pages with high counts may remain unnecessarily.
- Requires maintaining counters.

6. MFU (Most Frequently Used)

MFU replaces the page with the highest reference count.

Working

- Maintain a counter for each page.
- Replace the page having the maximum count.

Advantages

- Useful in some special situations where heavily used pages are unlikely to be needed again.

Disadvantages

- Generally less efficient than LRU.
- Rarely used in practical systems.

Comparison of Page Replacement Algorithms

Algorithm	Page Replaced	Advantage	Disadvantage
FIFO	Oldest page	Simple implementation	Suffers from Belady's Anomaly
Optimal	Page used farthest in future	Minimum page faults	Cannot be implemented practically
LRU	Least recently used page	Good performance	Requires additional hardware/software
MRU	Most recently used page	Useful in specific cases	Poor general performance
LFU	Least frequently used page	Keeps frequently used pages	Needs reference counters
MFU	Most frequently used page	Useful in special cases	Rarely used



SAQs:

1. Differentiate Logical Address and Physical Address.

Logical Address	Physical Address
Generated by the CPU.	Actual address in main memory.
Also called Virtual Address (during execution-time binding).	Loaded into the Memory Address Register.
Visible to the user program.	Not visible to the user program.
Translated by the MMU into a physical address.	Used by memory hardware to access data.

2. Differentiate Internal Fragmentation and External Fragmentation.

Internal Fragmentation	External Fragmentation
Wastage occurs inside an allocated partition.	Wastage occurs between allocated partitions.
Caused by fixed-size memory allocation.	Caused by variable-size memory allocation.
Unused space remains within the allocated block.	Free memory is divided into many small holes.
Cannot be removed by compaction.	Can be removed by compaction.

3. What is Swapping?

Swapping is a memory management technique in which a process is temporarily moved from main memory to backing store (disk) and later brought back into main memory for continued execution.

Advantages

- Increases degree of multiprogramming.
- Better memory utilization.

4. What is Compaction?

Compaction is the process of combining scattered free memory spaces into one large contiguous free block.

It is used to eliminate external fragmentation and is possible only when dynamic relocation is used.

5. What is Effective Access Time (Paging with TLB)?

Effective Access Time (EAT) is the average time required to access memory, considering both TLB hits and TLB misses.

A higher TLB hit ratio results in a lower Effective Access Time and better system performance.

6. Explain Valid–Invalid Bit.

Each page table entry contains a Valid–Invalid Bit.

- Valid Bit (V): Indicates that the page belongs to the process and can be accessed.

- **Invalid Bit (I):** Indicates that the page does not belong to the process. Accessing it generates a trap to the operating system.

The Valid–Invalid Bit provides memory protection.

7. Explain Sharing in Paging.

Paging allows multiple processes to share the same physical frame.

- Different page tables may point to the same frame.
- Shared pages are usually read-only.
- Sharing reduces memory usage and improves system efficiency.

8. What are the Disadvantages of Paging?

- Causes Internal Fragmentation.
- Requires additional memory for page tables.
- Address translation increases memory access time.
- Two memory accesses are required without TLB.
- Large page tables consume more memory.

9. Define Virtual Memory.

Virtual Memory is a memory management technique that allows the execution of processes without loading the entire program into main memory.

It separates logical memory from physical memory, allowing programs larger than the available physical memory to execute.

10. What is Locality of Reference?

Locality of Reference is the tendency of a process to access the same set of pages repeatedly during a particular period of execution.

It is the basis of the Working Set Model and helps reduce page faults.

11. Explain Page Fault.

A Page Fault occurs when the required page is not present in main memory.

The operating system:

1. Checks whether the reference is valid.
2. Locates the required page on disk.
3. Loads it into a free frame (or replaces another page).
4. Updates the page table.
5. Restarts the interrupted instruction.

12. What is the purpose of Dirty Page / Dirty Bit?

The Dirty Bit (Modify Bit) indicates whether a page has been modified after being loaded into memory.

- Dirty Bit = 1: The page has been modified and must be written back to disk before replacement.
- Dirty Bit = 0: The page has not been modified and need not be written back.

This reduces unnecessary disk write operations.

13. Explain Demand Paging.

Demand Paging is a virtual memory technique in which a page is loaded into main memory only when it is actually required.

If the required page is absent, a page fault occurs, and the operating system loads the page from disk into memory.

Advantages

- Better memory utilization.
- Faster program loading.
- Supports execution of large programs.

14. What is Thrashing?

Thrashing is a condition in which a process spends more time paging than executing due to insufficient memory frames.

Effects

- CPU utilization decreases.
- Page fault rate increases.
- System throughput decreases.
- Effective memory access time increases.

15. List the Counting Page Replacement Algorithms.

The counting-based page replacement algorithms are:

1. LFU (Least Frequently Used) – Replaces the page with the lowest reference count.
2. MFU (Most Frequently Used) – Replaces the page with the highest reference count.

16. What is the Memory Management Unit (MMU)?

The Memory Management Unit (MMU) is a hardware device that translates logical addresses generated by the CPU into physical addresses.

It performs address mapping dynamically and provides memory protection.

17. Differentiate Paging and Segmentation. (PYQ)

Paging	Segmentation
Divides memory into fixed-size pages and frames.	Divides memory into variable-size segments.
Eliminates external fragmentation.	May suffer from external fragmentation.
Programmer is unaware of paging.	Programmer is aware of segments.
Address = Page Number + Offset.	Address = Segment Number + Offset.

18. What is the 50% Rule? (PYQ)

The 50% Rule states that:

In a system using First Fit memory allocation, the amount of memory lost due to external fragmentation is approximately 50% of the total allocated memory.

This rule gives an estimate of memory wasted because of external fragmentation.

19. Differentiate Page Fault and Page Replacement. (PYQ)

Page Fault	Page Replacement
Occurs when the required page is not in main memory.	Process of replacing an existing page when no free frame is available.
Triggers the operating system to load the required page.	Selects a victim page using a page replacement algorithm.
May or may not require replacement.	Performed only when no free frame exists.